

FAST NUCES Karachi, campus



OS Lab Project Proposal

Maryam Saleem 24K-0779

Umaina Manzoor 24K-0501

Aiman Misbah. 24K-0842

CS-2006: Operating Systems
Project Proposal

Railway Signal Coordination System

1. Introduction

Modern railway systems rely on precise signal coordination to prevent collisions and ensure smooth train movement across shared track segments. When multiple trains attempt to access the same track simultaneously, improper synchronization may lead to resource conflicts, delays, or system deadlock.

This project implements a multi-threaded railway signaling simulator using Operating System synchronization mechanisms. Each train is modeled as an independent thread, while railway track segments act as shared resources. The system uses mutex locks to guarantee exclusive access to track segments and semaphores to manage signal permissions for safe train movement.

The project demonstrates key OS concepts such as thread synchronization, mutual exclusion, resource allocation, and deadlock prevention through controlled routing of trains across the track network.

2. Objectives

The main objectives of the project are:

- To simulate concurrent train movement using multi-threading.
- To manage shared track resources safely using synchronization primitives.
- To prevent deadlocks through safe routing and ordered resource allocation.
- To demonstrate semaphore-based signaling control.
- To visualize railway track usage and train movement through a graphical interface.

3. Methodology

The system will be implemented in C/C++ using multi-threading libraries and synchronization primitives.

System Architecture

The system consists of the following components:

Component	Description
Train Threads	Each train runs as an independent thread attempting to move across a sequence of track segments.
Track Segment Manager	Each track segment acts as a shared resource protected using mutex locks to ensure that only one train occupies it at a time.
Signal Controller	Semaphores control train entry into track zones and intersections, simulating real railway signal systems.
Deadlock Prevention	A consistent track acquisition order is enforced so that trains request resources in a fixed sequence, preventing circular wait conditions.

Implementation Steps

1. Define track segments and initialize mutex locks for each segment.
2. Create multiple train threads that follow predefined routes.
3. Implement semaphore-based signal control at critical track intersections.
4. Enforce ordered track locking to ensure deadlock-free routing.
5. Simulate train delays and scheduling behavior.
6. Log train movement and track allocation events.
7. Build a GUI to visualize tracks and train movement.

4. Mandatory Features

The following required components will be implemented:

- Train threads representing concurrent train movement
- Track segment locking using mutex
- Semaphore-based signaling system
- Deadlock-free routing mechanism

5. Additional Features

The project will include the following additional enhancements:

- Priority trains (e.g., express trains given preference)
- Delay simulation for realistic timing behavior
- System logging of train activities
- Graphical route visualization
- Throughput analysis and performance statistics

6. Expected Results

The final system will simulate multiple trains moving safely across shared railway tracks without collisions or deadlocks. The graphical interface will show real-time train movement and signal coordination, while logs and statistics will provide insight into system performance and synchronization efficiency.

7. Tools and Technologies

Category	Technology
Programming Language	C / C++
Threading Library	POSIX Threads (pthreads) or C++ threads
Synchronization	Mutex locks and Semaphores
GUI Framework	SFML / Qt / OpenGL / Raylib (for visualization)
Version Control	GitHub

8. Conclusion

This project provides practical experience with fundamental Operating System concepts such as multi-threading, synchronization, and deadlock prevention. By simulating railway signal coordination, the system demonstrates how OS-level resource management techniques can be applied to real-world infrastructure systems.